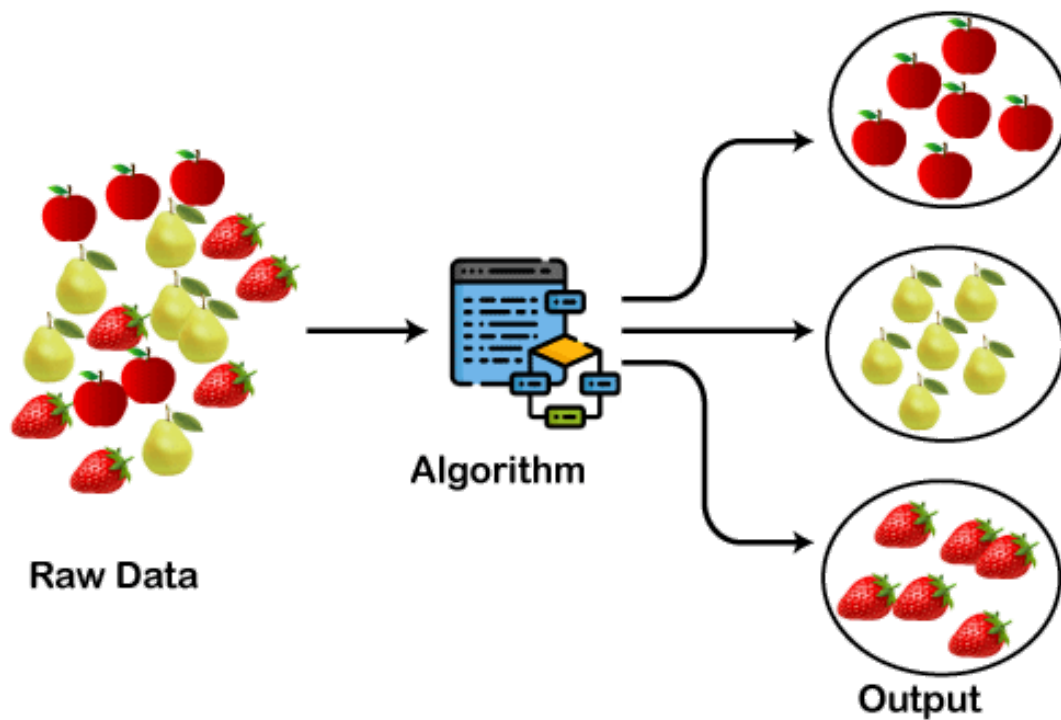


Report: Assignment 1

PCA & CLustering



Files Used

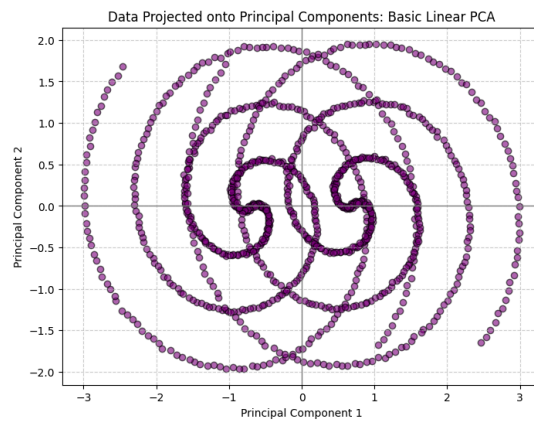
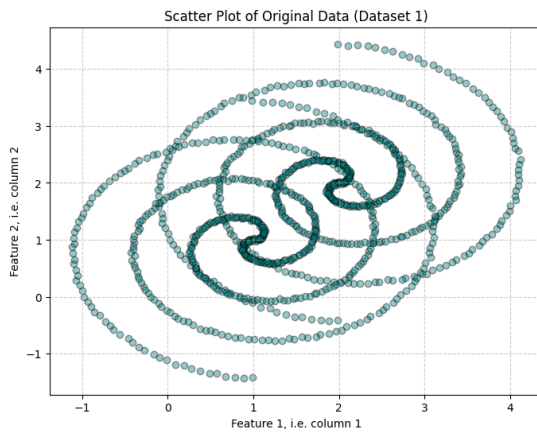
1. The problem statement and the Datasets:
 - a. *dataset1-assignment1 - Sheet1.csv*: The dataset for the first question
 - b. *Dataset2-Assignment 1 - Dataset.csv*: The dataset for the second question
 - c. *FML_assignment_1_2025.pdf*: the problem statement
 2. *Pca_solution_q1.ipynb*: Code solution for the first question
 3. *Kmeans_solution_q2.ipynb*: Code solution for the second question
-

Problem 1

Variance Explained by Linear PCA

The following are the results from the code notebook: Below is a table that describes the weight loading of the linear PCA, as well as the variance explained by each of them.

Principal Component	Principal Component 2
Variance Explained: 65.39%	Variance Explained: 34.61%
Feature weighting: Feature 1: -0.6910 Feature 2: -0.7228	Feature weighting: Feature 1: -0.7228 Feature 2: 0.6910



Kernel PCA

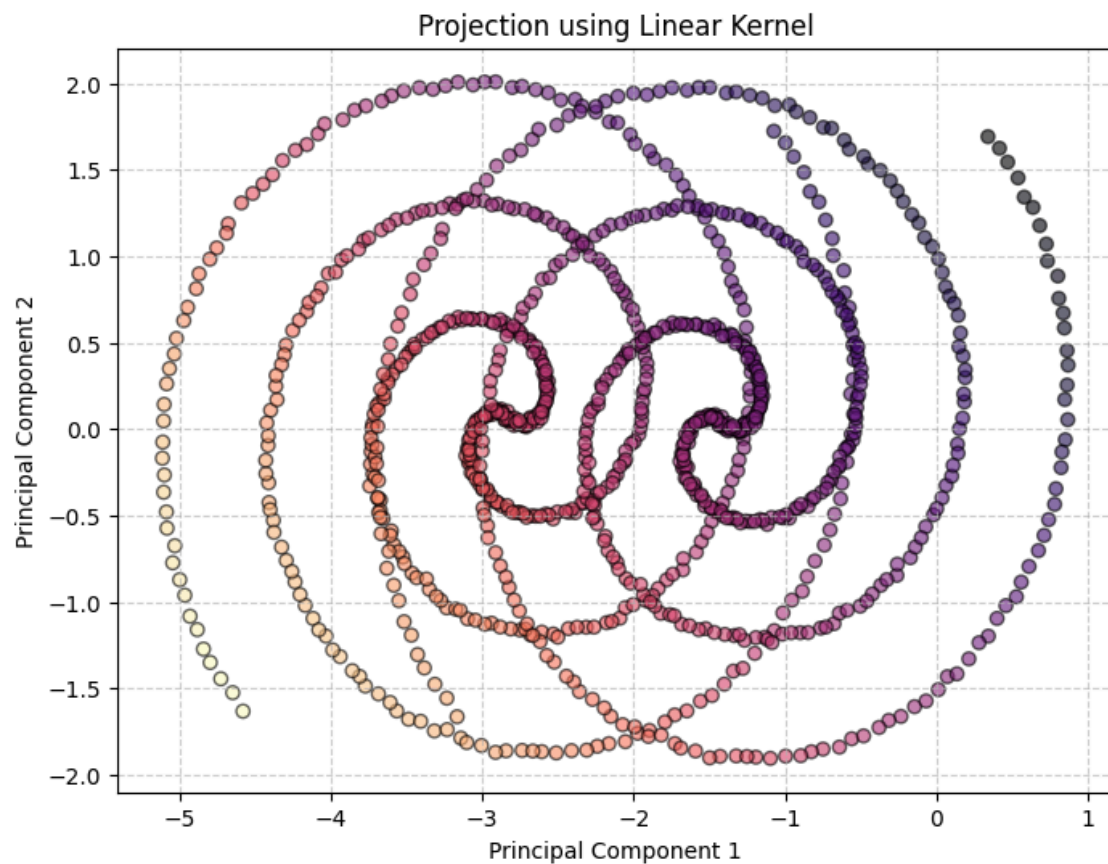
Types of Kernels Used:

1. Linear Kernel
2. Polynomial Kernel
3. Gaussian Kernel, i.e. The Radial Basis

Linear Kernel

This is the same as normal PCA and has been used as a sanity check:

- The ratio of explained variance by the PCs is the same
- The datapoints are still not linearly separable



Polynomial Kernel

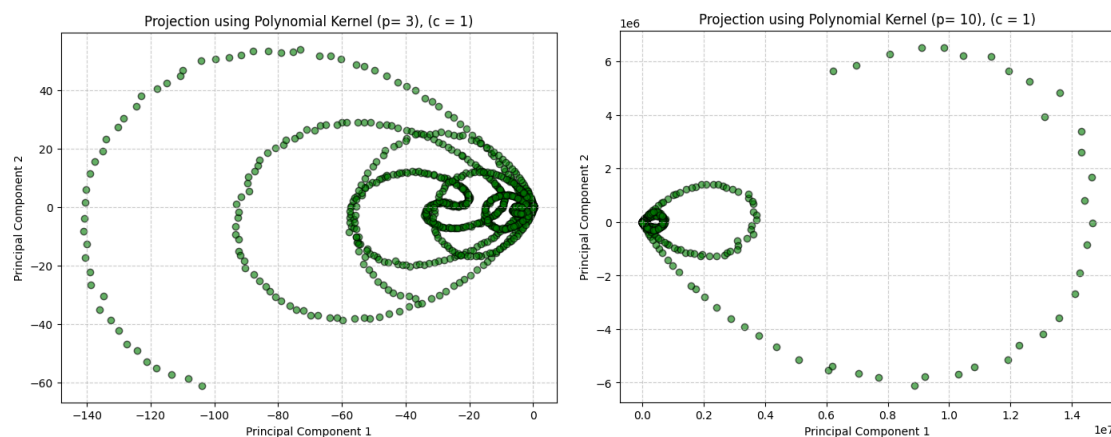
The kernel function is given by: $K(x, y) = (x \cdot y + c)^p$

In the image below, I have used the parameters $(c, p) = (1, 3)$ and $(1, 10)$

However, the hyperparameters can be swept to find the optimum (c, p) that achieves separability of the data points in the PC space.

The best degree for the polynomial kernel in my analysis is 10, as it provides the best separation between the options tested.

The degrees (like 2 and 3) begin to separate the data, but they are not powerful enough to fully untangle the spirals, leaving significant overlap.



Radial Basis Function Kernel

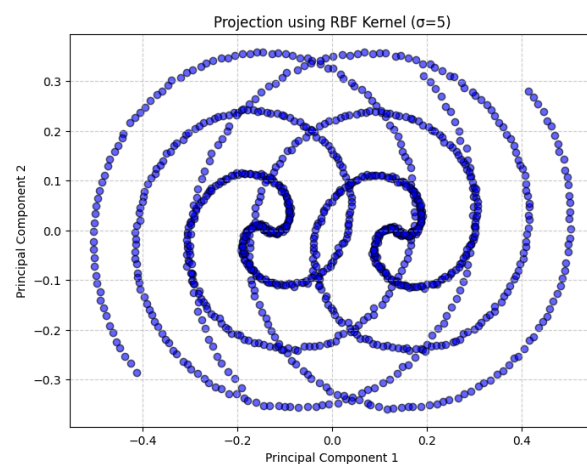
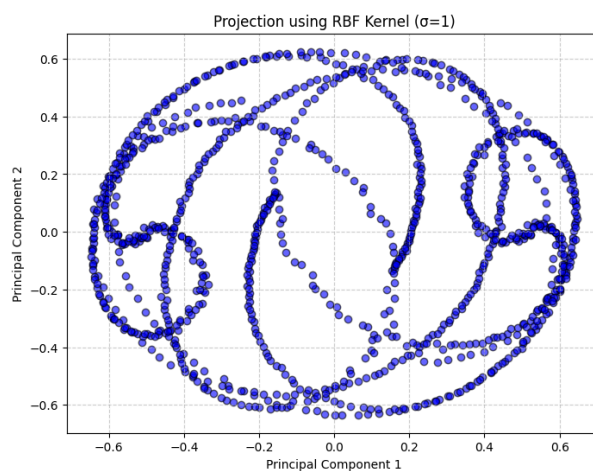
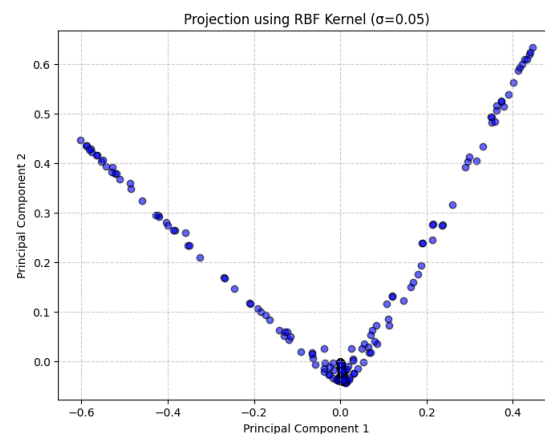
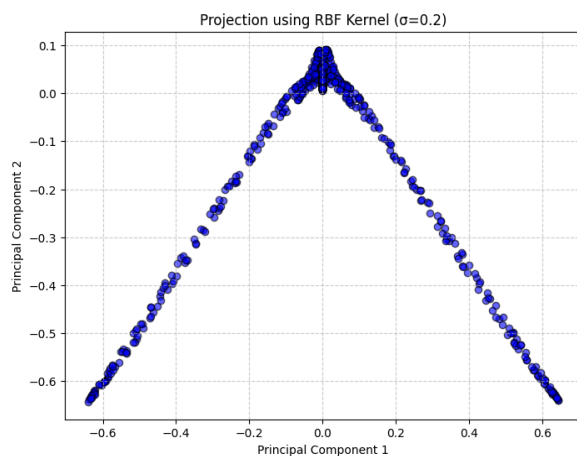
In the RBF kernel, the sigma value (σ) is a hyperparameter that controls the width or scale of the kernel function, determining how quickly the similarity score drops with distance.

A smaller sigma value results in a more peaked, local kernel that only considers nearby points as similar, leading to a more complex, non-linear decision boundary.

We observe that as sigma increases, the separation decreases and becomes more similar to linear PCA.

Analysis of sigma values:

- Too high sigma values (1,5,10):
 - When sigma is large, the RBF kernel behaves like a linear kernel and the data remains inseparable
- Too low a sigma (e.g., 0.05):
 - A very small sigma can be too sensitive to local noise and may not capture the overall structure of the spirals.
- Just right ($\sigma = 0.2$):
 - This value provides the best result, untangling the two spirals and projecting them into two distinct, linearly separable clusters. This is the ideal outcome.



The Best Kernel

The RBF kernel is the best suited for this dataset.

Reasoning:

While a high-degree polynomial is also able to separate the data, the RBF kernel is a more natural fit for this problem.

The visual clarity of the separation achieved with the RBF kernel (at $\sigma=0.2$) is superior to that of the polynomial kernel.

The dataset consists of two classes defined by the local proximity of their points (i.e., points in a spiral are close to other points in the same spiral).

The RBF kernel's operation is based on local influence, where the sigma parameter defines the "neighbourhood" of this influence. This allows it to effectively capture the structure of the intertwined spirals and pull them apart cleanly.

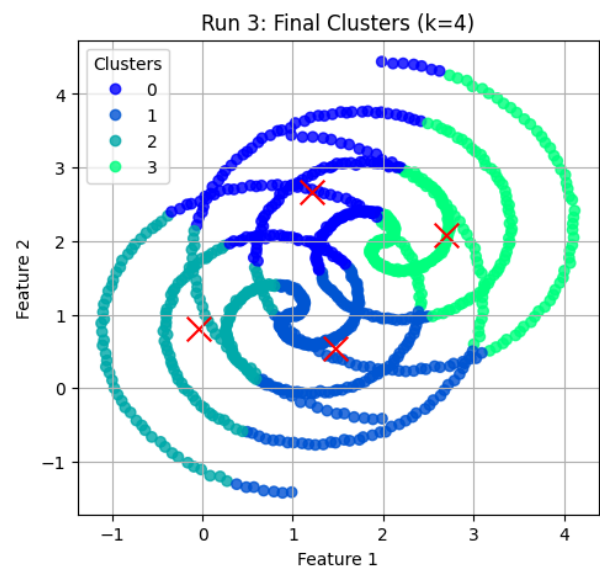
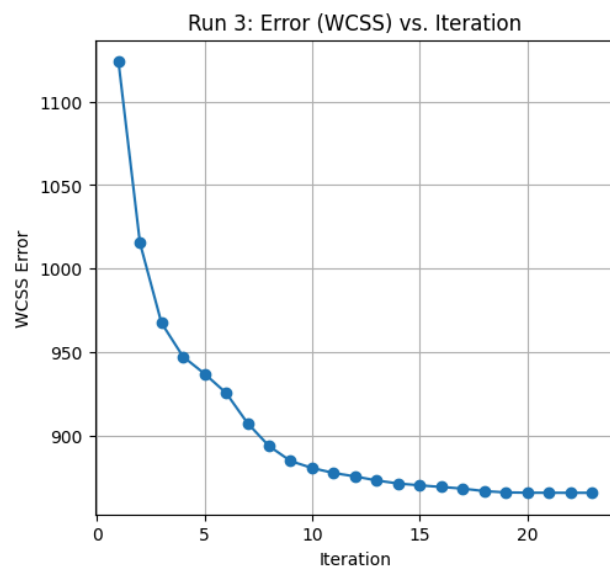
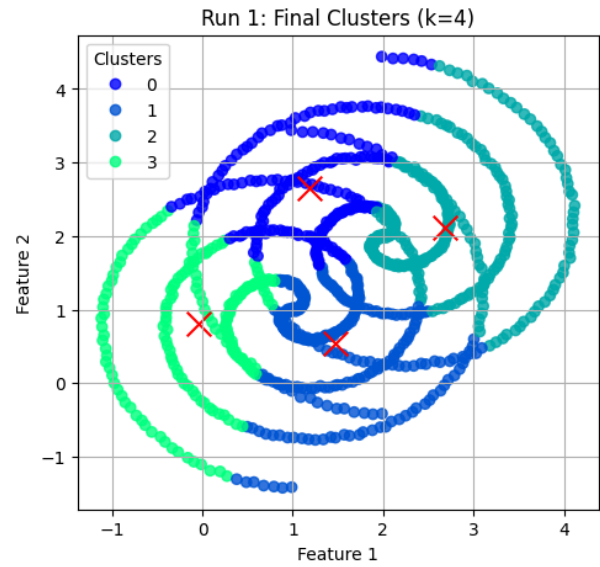
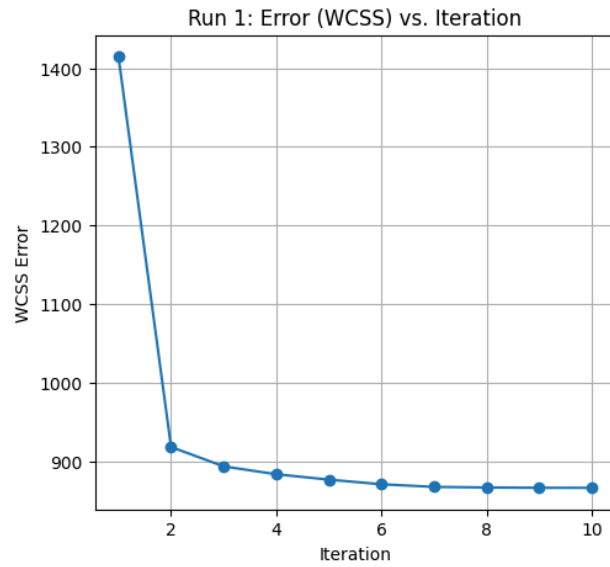
The polynomial kernel, on the other hand, tries to fit a global polynomial function to the data, which can also work but is less intuitive and potentially less robust for this specific geometric structure.

Problem 2

Plot of the Within-Cluster Sum of Squares (WCSS) and the Final Clustering

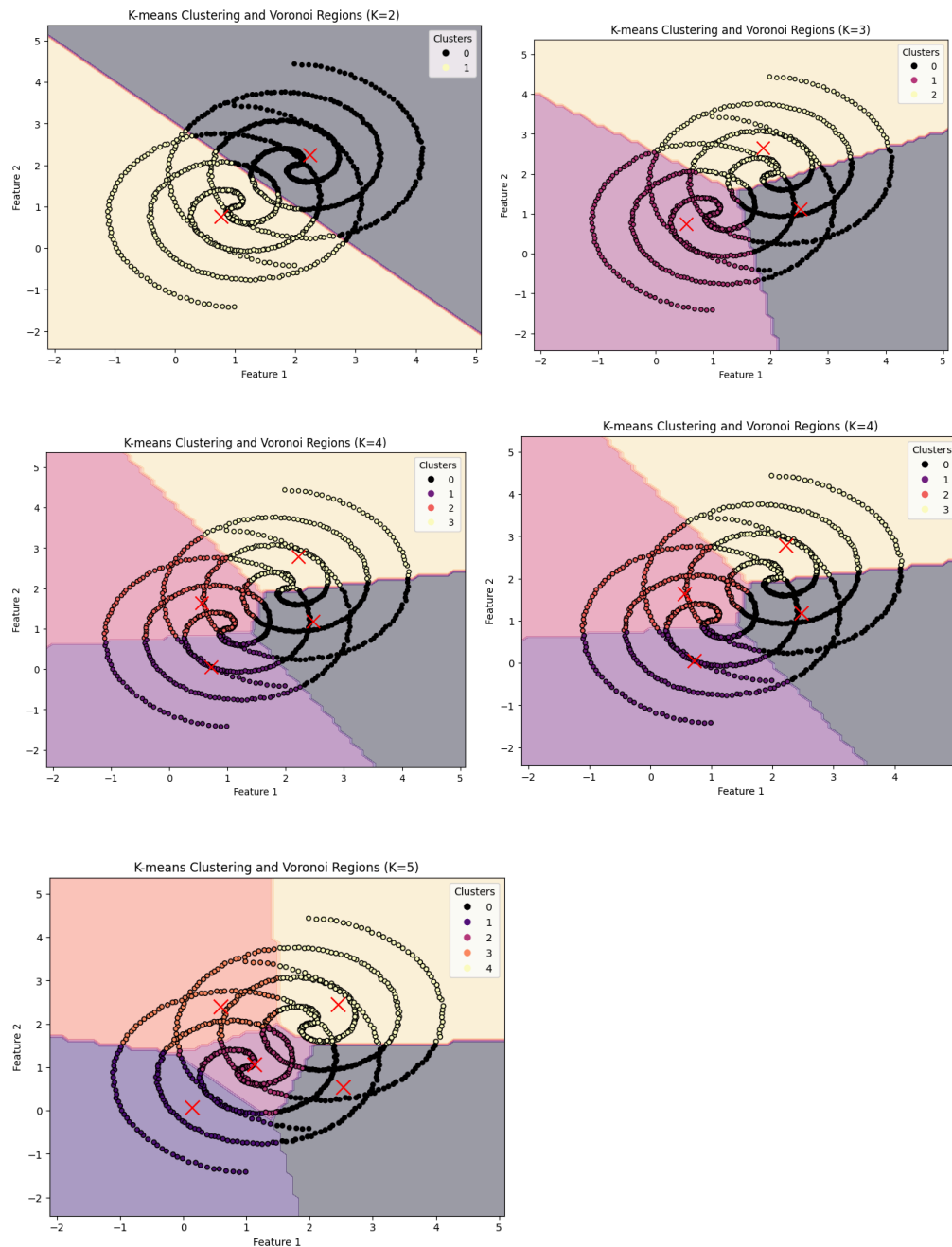
We see that the WCSS decreases differently for each initialisation. However, the final clustering seems quite similar and the WCSS converge to a minimum for each initialisation.

I have attached 2 of the 5 plots to illustrate this point.



Plotting the Voronoi Regions

I used a random seed of 42 for all the initialisations to ensure reproducibility.

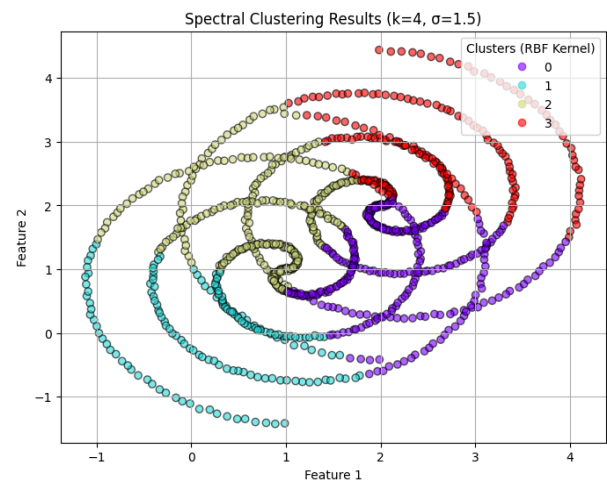
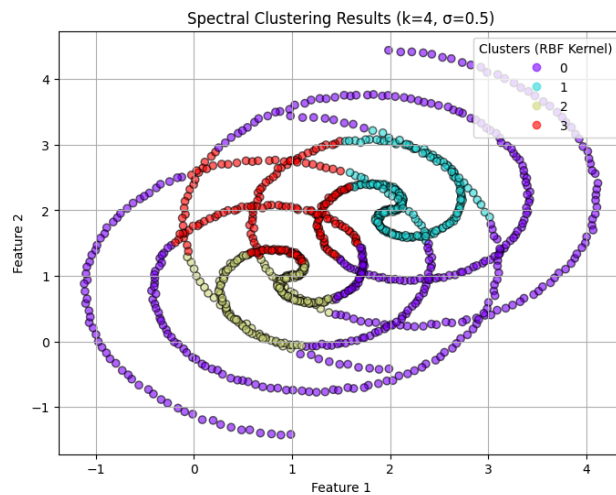


Spectral Clustering

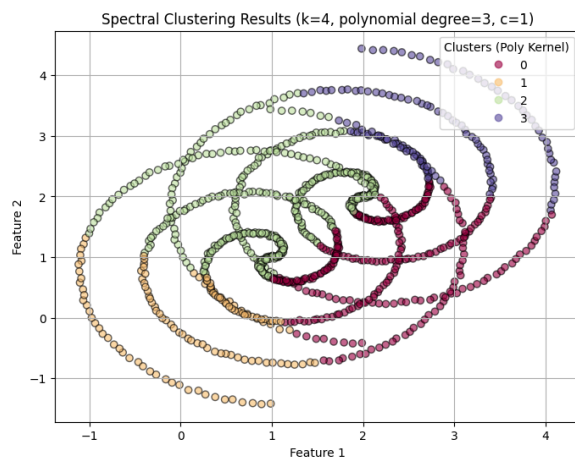
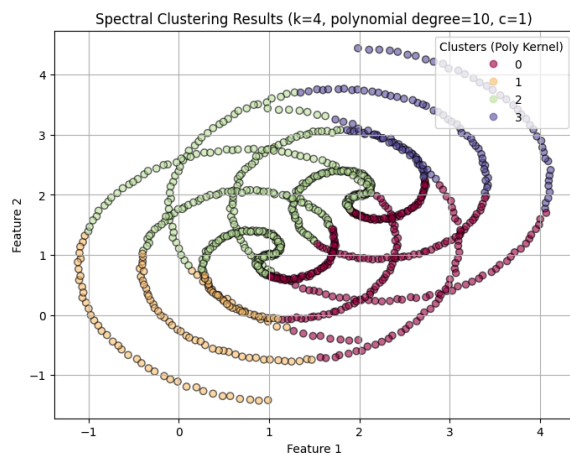
I used the polynomial and RBF kernels for the spectral clustering.

RBF Kernel

As sigma increases, the clustering approaches the normal K-means clustering. Here, a higher sigma performs better than a very low one.

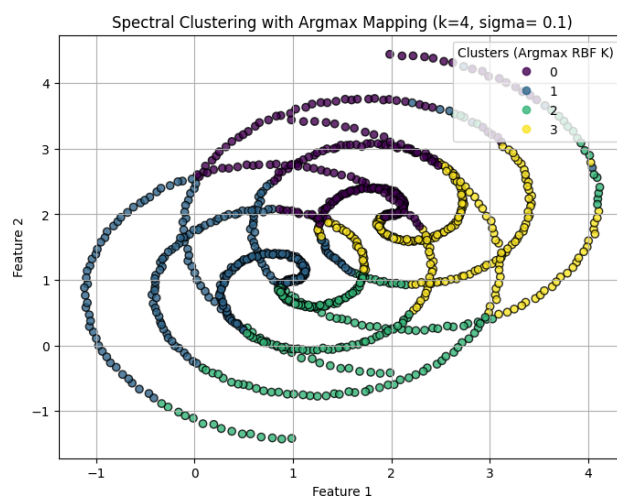


Polynomial Kernel



There doesn't appear to be much difference between the clusters with different degrees. This doesn't necessarily mean a linear kernel is the "best," but it strongly implies that a linear kernel is good enough since the clusters are already separable as linear blobs.

Clustering Performance With SpectralClustering_argmax



The top right of the final clustering with this approach has some points from cluster 2 that are visually assigned to the wrong cluster!

This is not surprising, as it is a result of the fundamental difference in how the cluster assignment is made in this method as compared to the regular RBF.

The Standard Method (K-means on Eigenvectors)

Standard spectral clustering takes the first k eigenvectors to create a new, D -dimensional space. Each data point is represented by a new set of coordinates derived from these eigenvectors. It then runs K-means on these new coordinates.

This process transforms the data so that complex, non-linear clusters in the original space become simple, linearly separable clusters in the new "spectral" space. Thus, K-means is very effective at finding separations in this new space as the clusters can be separated by

Voronoi regions. It uses the complete information from all k eigenvectors for each point to understand its relationship to other points.

The argmax Method

This method takes the k eigenvector values for each data point and simply assigns the point to the cluster corresponding to the eigenvector with the highest value.

This method is less effective as it discards a lot of crucial information. Instead of using the k eigenvector values as a rich new set of coordinates, it only pays attention to which one is the largest. This loses the subtle relationships between the coordinates that the standard K-means approach uses to find the optimal cluster boundaries. The assignment is very "hard" and direct, without the iterative refinement that K-means provides.